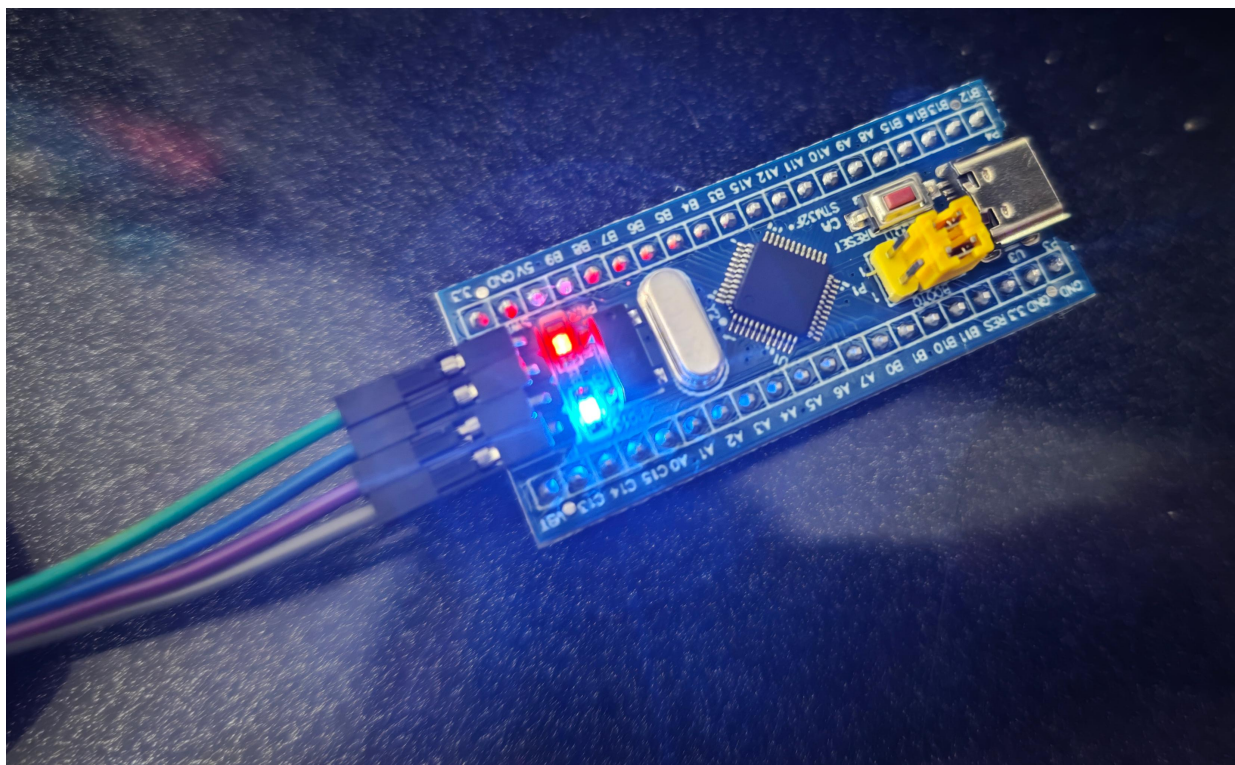


电控第一次作业 说明文档

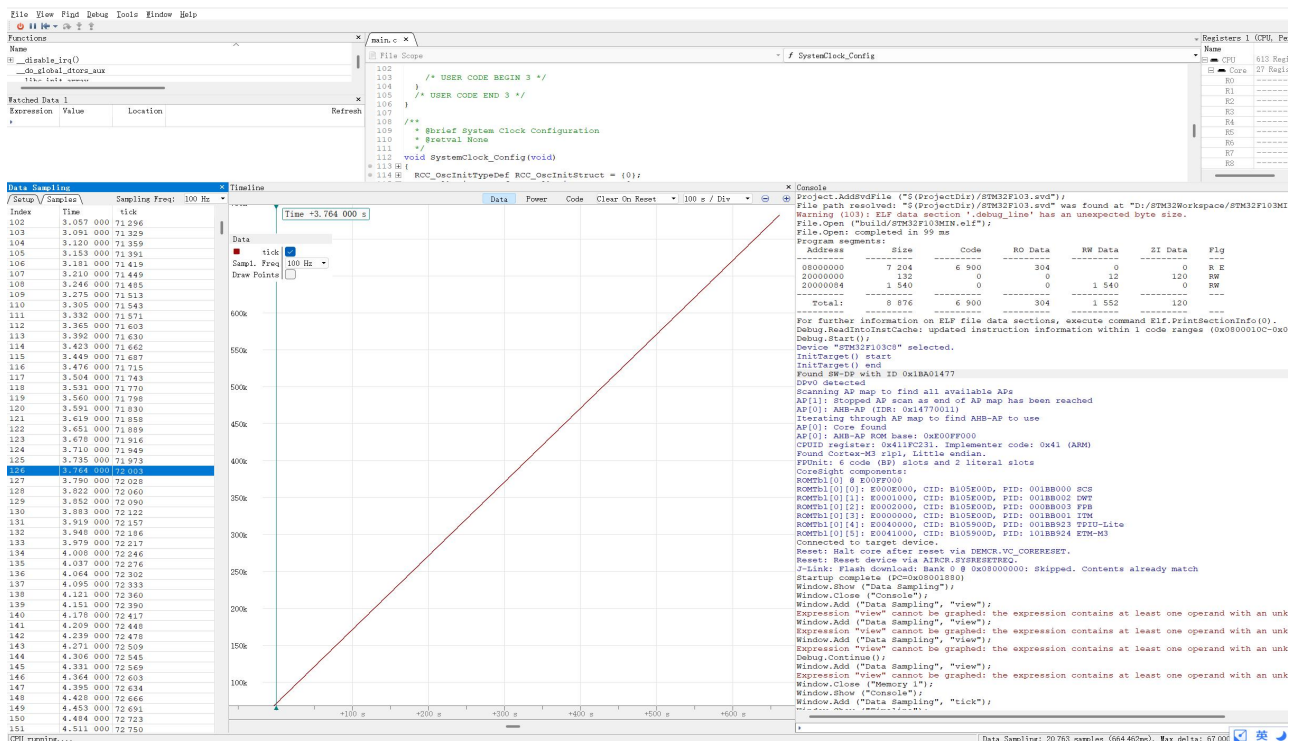
姓名：梁昕禾 学号：3250100274

引入：交付图片

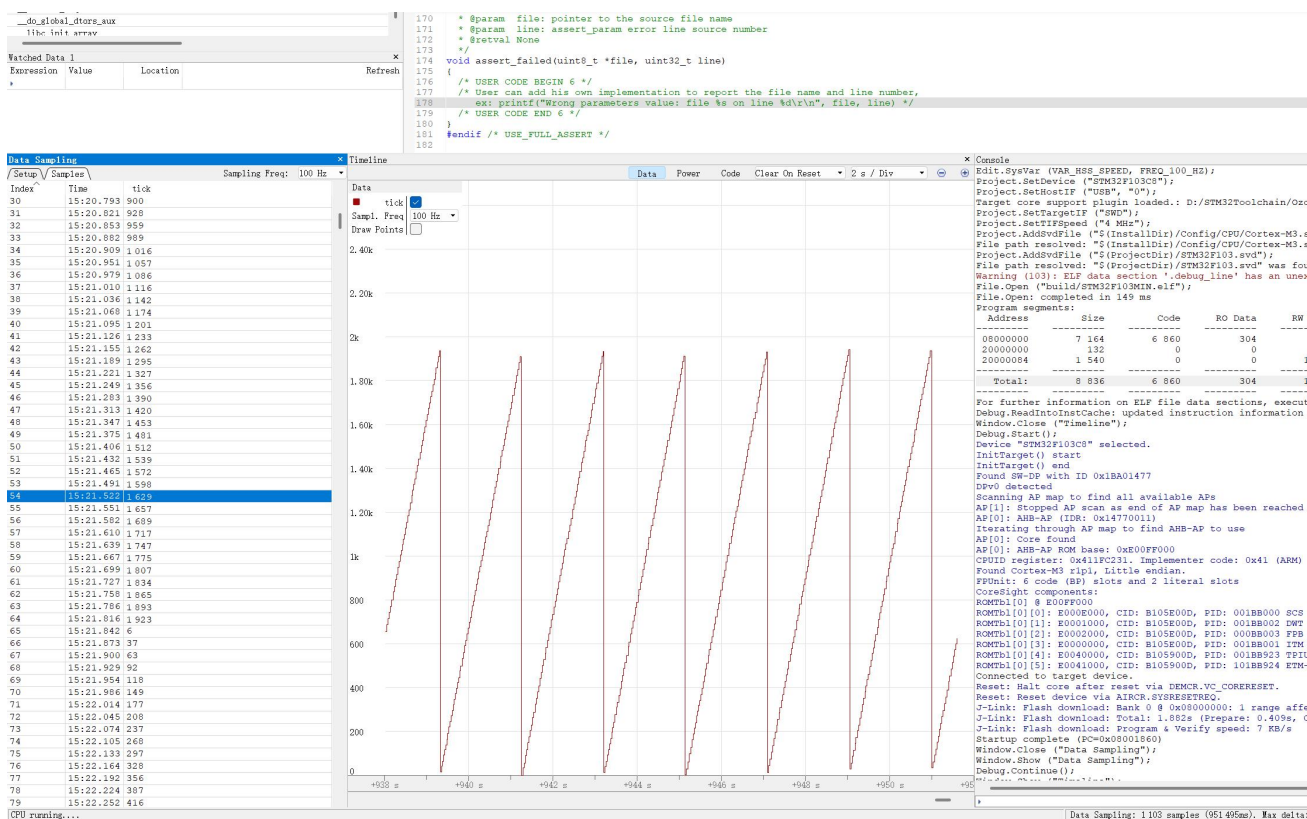
1. GPIO：附灯亮的照片。

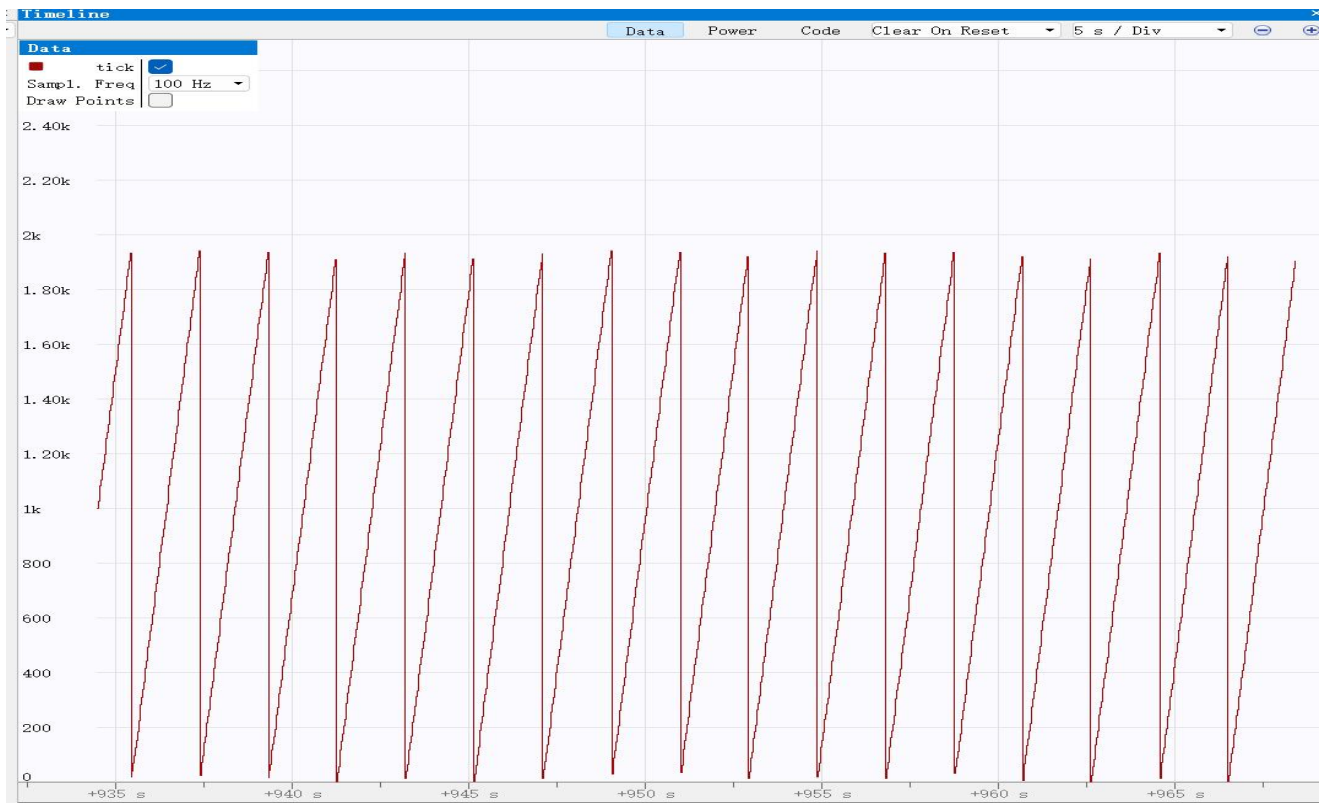


2. 定时器： tick 采样，0s 至 600 s 内由 0 增至 617.1k，每秒 1000 左右



3. 看门狗：tick tick 波形，涨至约 1900 后归零并不断重复





正文

一、环境与工程组织

本次作业使用 STM32CubeMX 生成工程，编译工具链为 CMake 配合 Ninja 与 GNU Arm GCC，调试使用 Ozone。构建类型选 Debug，以保证变量不被优化掉，调试时能看到实时值。在 CubeMX 中把 Toolchain/IDE 设为 CMake 并勾选 Generate Under Root，生成之后把 CMakeLists.txt 换成队内发放的模板，其中 `set(user_folders "Tasks")` 保持不动。同时要把 SYS 里的 Debug 设为 Serial Wire，否则生成代码会关闭 SWD 引脚，调试器无法连接。

工程根目录下需要在执行 CMake Configure 之前就建好 Tasks/inc 与 Tasks/src，因为模板是用文件通配的方式收集源文件的，目录不存在时 Tasks 不会进入编译。三题做在同一个工程里，业务代码只写在 Tasks 下，main.c 中只在 USER CODE 段包含头文件并调用一次初始化函数，while(1) 保持为空。这样划分之后，每次重新生成代码都不会冲掉自己写的部分。定时器的更新回调全工程只保留一份，放在 Tasks 的源文件中，因为若 Core 中另有一份同名回调，链接时会报重复定义。

调试侧使用 Ozone，通过新建工程向导依次选定器件 STM32F103C8、寄存器组 Cortex-M3 以及对应的 SVD 文件，接口选 SWD，最后指定 build 目录下的 .elf 文件。Ozone 默认的

连接方式是下载并复位运行，因此每次开始调试都会把当前的 elf 重新烧写一遍；如果日志里显示 Flash download 被跳过、内容与目标一致，说明二进制没有变化，这时若刚改过代码，就要回头检查 elf 是否真的重新编译过。

二、定时器周期的推导

定时器挂在 APB1 总线上，因此计算周期之前必须先确定它的时钟频率。外部晶振 HSE 为 8 MHz，经 PLL 九倍频后得到系统时钟：

$$\text{SYSCLK} = \text{HSE} \times 9 = 8 \text{ MHz} \times 9 = 72 \text{ MHz}$$

AHB 预分频为 1，故 HCLK 同样为 72 MHz；APB1 预分频为 2，PCLK1 为 36 MHz。这里有一处容易算错的地方：当 APB1 的预分频系数不为 1 时，接在 APB1 上的定时器时钟按 PCLK1 的两倍计算，即

$$\text{TIMxCLK} = \text{PCLK1} \times 2 = 36 \text{ MHz} \times 2 = 72 \text{ MHz}$$

若 APB1 的分频系数为 1，则定时器时钟等于 PCLK1，不存在这个二倍关系。确定定时器时钟之后，周期由预分频寄存器 PSC 和自动重装寄存器 ARR 共同决定：

$$\frac{(\text{PSC} + 1) \times (\text{ARR} + 1)}{\text{TIMxCLK}} = T$$

取 PSC = 71，则计数时钟为

$$\frac{72 \text{ MHz}}{71 + 1} = 1 \text{ MHz}$$

也就是每计数一次代表 1 μs。要得到 1 ms 的周期，需要

$$\text{ARR} + 1 = \frac{1 \text{ ms}}{1 \mu\text{s}} = 1000$$

故 ARR = 999。这里先定 PSC 再定 ARR 是有顺序的：先让分频后的计数时钟落在 1 MHz 这样的整数上，每次计数正好对应 1 μs，自动重装值就只需要等于以微秒为单位的周期长度，计算和核对都方便；如果先把 ARR 定死，PSC 往往要取非整数，反而不好验算。把两个数值代回原式：

$$\frac{(71 + 1) \times (999 + 1)}{72\,000\,000} = \frac{72\,000}{72\,000\,000} = 1.000000 \times 10^{-3} \text{ s}$$

结果正好是 1 ms，说明中断每秒触发 1000 次。回调中 tick 每次加一，因此 tick 的增长速率即为每秒 1000。tick 需要声明为全局的 volatile uint32_t，volatile 的作用是要要求编译器每次访问都从内存读取，否则在开启优化后它可能被保存在寄存器中，调试器监视到的就不是实时值。

三、看门狗超时的推导

独立看门狗由内部低速时钟 LSI 驱动，与 HSE、PLL 构成的主时钟没有关系。这一点在调试时很关键：CPU 被暂停时 LSI 仍在振荡，看门狗继续计数，因此在断点上停留超过超时时间，芯片就会复位、调试器随之掉线。作业要求观察时保持程序运行、不要停在断点上，原因正在这里。超时时间的计算式为

$$\text{超时} = \frac{(\text{Reload} + 1) \times \text{预分频}}{\text{LSI}}$$

F103 的 LSI 按手册约 40 kHz。取预分频 64、Reload 为 1249，代入得

$$\frac{(1249 + 1) \times 64}{40\,000} = \frac{80\,000}{40\,000} = 2.0\text{ s}$$

超时时间必须明显大于 1 ms 的中断周期。第 3 题去掉喂狗之后，芯片每 2 s 复位一次，而 tick 每秒增加 1000，每个周期内 tick 大约能涨到 2000，从而形成一个完整的锯齿；若超时接近 1 ms，tick 还没来得及增加就会被复位清掉，现象便不成立。实测的复位周期约 1.9 s、峰值约 1900，与理论值 2.0 s、2000 的偏差来自 LSI 实际频率与标称值的差异，属于正常范围。

四、三题的实现、现象与切换方式

三题共用同一份源码，通过 Tasks/src/main_task.c 顶部的宏选择，取值 1、2、3 分别对应 GPIO、定时器、看门狗三题，改动之后重新编译下载即可。

```
#define HW_TASK 1
```

这里采用条件编译而不是把代码注释掉，是因为注释需要人工记住开关位置，改动多了容易遗漏；宏在预处理阶段就决定了哪些代码参与编译，还可以配一条 #error 拦住非法取值。当 HW_TASK 不等于 2 时，喂狗语句 HAL_IWDG_Refresh 位于 #if HW_TASK == 2 条件之内，根本不会编译进程序。

第 1 题只把 PC13 配置为推挽输出并写入低电平，板载 LED 因低电平而点亮；此时不启动定时器，也就没有人喂狗，芯片约每 2 s 复位一次，因此观察需要在复位之前完成。第 2 题启动 TIM2 的更新中断，回调中让 tick 自增并在同一处调用 HAL_IWDG_Refresh，看门狗按时喂上，芯片不复位，tick 稳定地每秒增加 1000。第 3 题只需在第 2 题的基础上去掉喂狗，其余配置不动，tick 便从 0 涨到约 2000 后因复位归零，随后不断重复。

五、调试过程中的几点体会

验证第 2 题时，起初把 Ozone 的采样频率设为 100 Hz，以为相邻两行样本相隔 10 ms，直接用行数估算了增长速率，得到的结果与理论值对不上。后来改为直接读取采样表格的 Time 列，才发现相邻样本的实际时间戳间隔约为 29 ms，也就是实际采样率只有约 34 Hz，与设定值差得较多。原因是所用调试器没有硬件 BMA 支持，Ozone 通过模拟方式采样，存在额外开销，跟不上设定的频率。这说明判断信号速率应当以时间戳为准，不能相信界面上设定的采样率。

另一个问题出在 Timeline 窗口的横轴刻度上。默认状态下 Time/Div 达到 100 s，而采集到的数据只有一两秒，波形被压缩在靠近原点的一小段里，几乎看不出形状；把横轴缩小到秒级之后，第 2 题的直线斜率和第 3 题的锯齿才清晰地显示出来。

由此也体会到，观察变量增速不能只看 Watched Data 里一个静止的数值，它只能说明当前值，不能说明变化速率；作业中特别强调这一点是有道理的，带时间戳的采样表格和波形图才是合适的证据。

用条件编译区分三题还带来一个便利：宏只影响编译结果，不影响工程结构，所以三题可以共用同一份 CubeMX 配置和同一份 CMakeLists.txt，切换成本很低。反过来说，每切换一次都必须重新编译并重新下载，否则 Ozone 打开的仍然是从前的 elf，看到的现象与当前源码对不上。第 1 题由于不喂狗，调试时若在断点上停留，芯片会在超时后复位，这一点和第 3 题是同一种情况。